

PLANNED

Production Architecture Specification

Family Savings & Allowance Platform

Version 1.0 | June 2026 | Staff/Principal Engineering

This document specifies the production infrastructure layer for the Planned family finance platform — designed to support 10,000+ families, 50,000+ children, and millions of transactions.

Contents

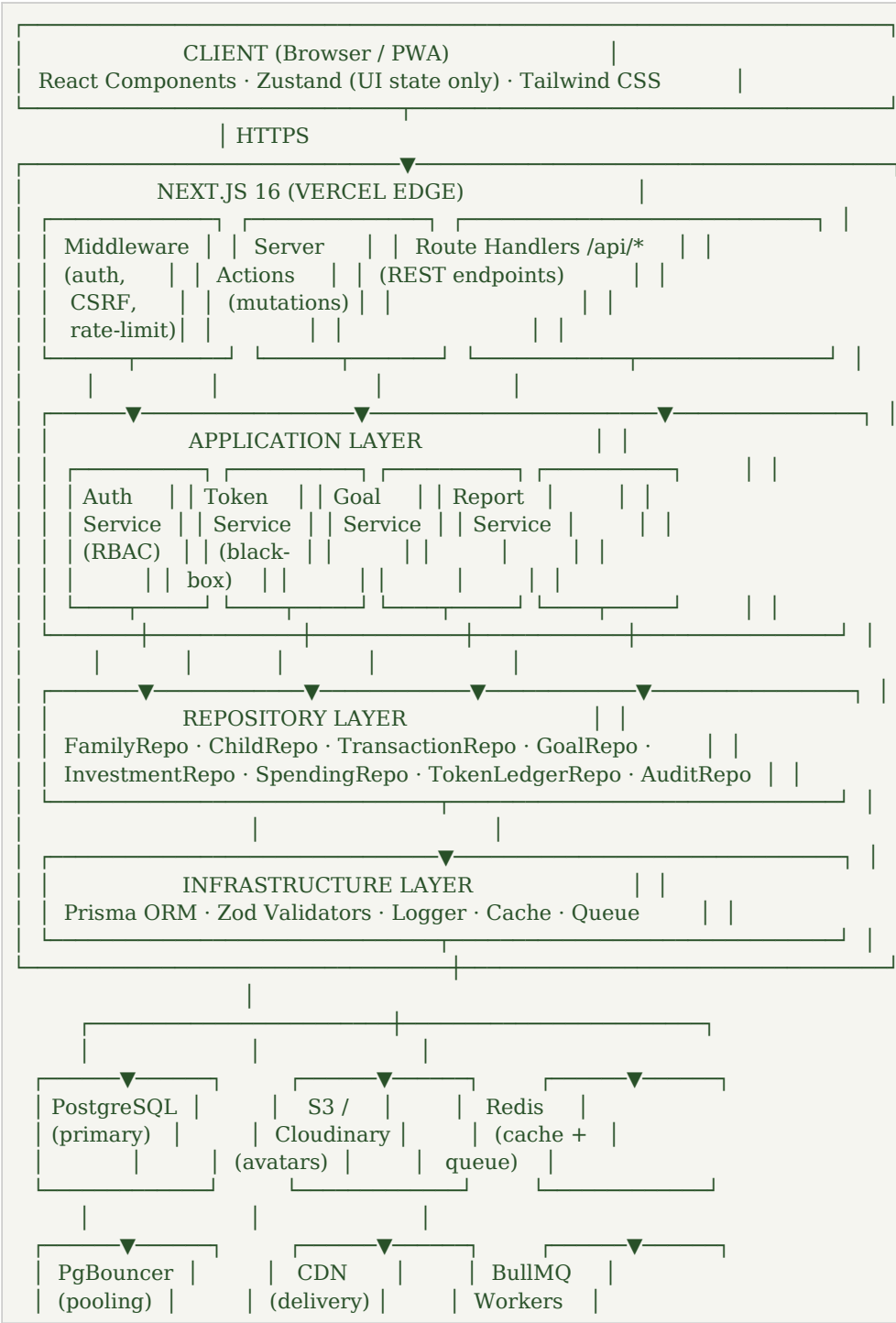
1. System Architecture Diagram
2. Folder Structure
3. Database Schema
4. Authentication Architecture
5. Authorization Architecture
6. API Architecture
7. State Management Architecture
8. Services Layer
9. Repository Layer
10. Notifications System
11. Scheduler System
12. Reporting System
13. File Upload System
14. Audit Trail System
15. Security Architecture
16. Deployment Architecture
17. Environment Variables
18. Migration Plan
19. Scaling Strategy
20. Implementation Roadmap

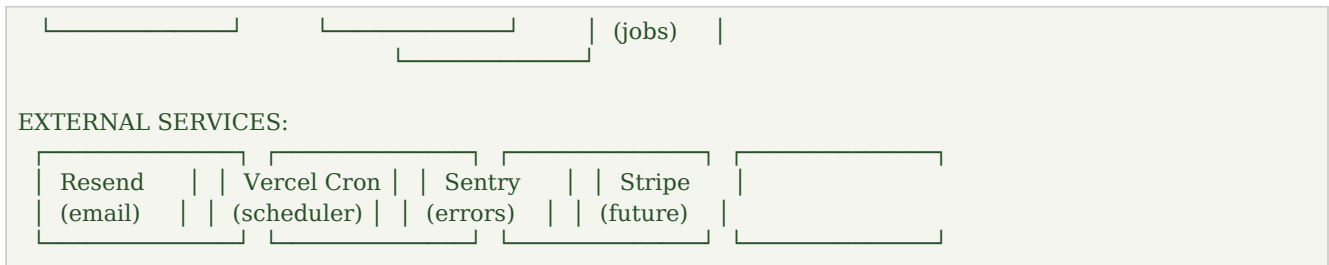
1. System Architecture Diagram

Overview

Planned is a multi-tenant family finance platform built on Next.js 16 with a clear separation between the presentation layer (React components), the application layer (server actions + route handlers), the domain layer (services + repositories), and the persistence layer (PostgreSQL + object storage). The architecture follows Clean Architecture + DDD principles so that business rules are isolated from infrastructure concerns.

High-Level Architecture (Text Diagram)





Reasoning

- **Separation of concerns:** The presentation layer contains ZERO business logic — components only render state and dispatch intents.
- **Dual API surface:** Server Actions handle mutations (type-safe, no manual fetch); Route Handlers handle queries + webhooks + file uploads.
- **Layered isolation:** Services own business rules (TokenService is a black-box interface); Repositories own data access; Infrastructure owns cross-cutting concerns.
- **Polyglot persistence:** PostgreSQL for transactional data, S3/Cloudinary for blobs, Redis for cache + job queue. Each store is best-in-class for its workload.
- **Serverless-first:** Vercel handles auto-scaling + edge caching. Background jobs run on Vercel Functions triggered by Vercel Cron, with BullMQ + Redis for heavy processing.

Tradeoffs

- **Serverless vs. containers:** Pro: simpler deployment, no container orchestration. Con: 60s function timeout on Vercel — long-running reports must be queued.
- **Server Actions vs. REST:** Pro: type-safe, ergonomic. Con: harder to cache than GET endpoints. Mitigation: use React Query for client-side caching of server-action reads.
- **PostgreSQL vs. DynamoDB:** Pro: familiar, excellent DX. Con: 10K families × 5 members × 1000 transactions = 50M rows — needs partitioning by year + familyId. Handled in Section 19.

2. Folder Structure

Production folder structure organized by feature + layer:

```
planned/
├── prisma/
│   ├── schema.prisma      # Full production schema (Section 3)
│   ├── migrations/        # Versioned SQL migrations
│   └── seed.ts            # Demo data seeder
├── src/
│   ├── app/               # Next.js App Router
│   │   ├── (auth)/        # Auth route group (login, register, forgot)
│   │   │   ├── login/page.tsx
│   │   │   ├── register/page.tsx
│   │   │   └── forgot-password/page.tsx
│   │   ├── (dashboard)/   # Protected route group
│   │   │   ├── layout.tsx  # Auth guard + sidebar shell
│   │   │   ├── page.tsx    # Parent overview
│   │   │   ├── children/page.tsx
│   │   │   ├── goals/page.tsx
│   │   │   ├── transactions/page.tsx
│   │   │   ├── investments/page.tsx
│   │   │   ├── tokens/page.tsx
│   │   │   ├── reports/page.tsx
│   │   │   └── settings/page.tsx
│   │   ├── (child)/       # Child-scoped route group
│   │   │   ├── layout.tsx  # Child auth guard
│   │   │   └── [childId]/page.tsx
│   │   ├── api/           # Route Handlers (REST)
│   │   │   ├── auth/[...nextauth]/route.ts
│   │   │   ├── families/route.ts
│   │   │   ├── children/route.ts
│   │   │   ├── goals/route.ts
│   │   │   ├── transactions/route.ts
│   │   │   ├── uploads/route.ts # Presigned URL generation
│   │   │   ├── reports/route.ts
│   │   │   └── webhooks/    # External webhooks
│   │   │       └── resend/route.ts
│   │   ├── layout.tsx     # Root layout (fonts, providers)
│   │   └── globals.css    # 4-theme design system
│   ├── components/        # Shared UI components
│   │   ├── ui/            # shadcn/ui primitives (button, card, etc.)
│   │   ├── charts/        # Editorial SVG charts
│   │   ├── modals/        # Shared modals
│   │   └── providers/     # React context providers
│   │       ├── auth-provider.tsx
│   │       ├── theme-provider.tsx
│   │       └── query-provider.tsx
│   ├── features/          # Feature modules (DDD)
│   │   ├── auth/
│   │   │   ├── components/ # LoginForm, RegisterForm
│   │   │   ├── actions.ts  # Server actions: login, register, logout
│   │   │   ├── schemas.ts  # Zod validation schemas
│   │   │   └── types.ts
│   │   ├── family/
│   │   │   ├── components/
│   │   │   └── actions.ts
```

- └─ schemas.ts
- └─ types.ts
- └─ children/
 - └─ components/
 - └─ actions.ts
 - └─ schemas.ts
 - └─ types.ts
- └─ transactions/
 - └─ components/
 - └─ actions.ts # save, withdraw, invest, redeem
 - └─ schemas.ts
 - └─ types.ts
- └─ goals/
 - └─ components/
 - └─ actions.ts # create, update, delete, contribute
 - └─ schemas.ts
 - └─ types.ts
- └─ investments/
 - └─ components/
 - └─ actions.ts
 - └─ schemas.ts
 - └─ types.ts
- └─ spending/
 - └─ components/
 - └─ actions.ts
 - └─ schemas.ts
 - └─ types.ts
- └─ tokens/ # Token economy (BLACK-BOX)
 - └─ components/ # UI only — no business logic
 - └─ actions.ts # Calls TokenService interface
 - └─ types.ts
 - └─ README.md # "DO NOT modify token calculations"
- └─ reports/
 - └─ components/
 - └─ actions.ts # generate, download, email
 - └─ types.ts
- └─ notifications/
 - └─ components/ # Bell dropdown, toast
 - └─ actions.ts
 - └─ types.ts
- └─ uploads/
 - └─ components/ # AvatarUploader
 - └─ actions.ts # requestPresignedUrl, confirmUpload
 - └─ types.ts
- └─ lib/ # Shared utilities (cross-feature)
 - └─ db.ts # Prisma client singleton
 - └─ auth.ts # NextAuth configuration
 - └─ validations.ts # Shared Zod schemas (email, password)
 - └─ errors.ts # Typed error classes
 - └─ logger.ts # Structured logger (pino)
 - └─ cache.ts # Redis cache wrapper
 - └─ rate-limit.ts # Rate limiter (Redis-backed)
 - └─ crypto.ts # Password hashing, token generation
 - └─ currency.ts # Multi-currency formatter
 - └─ utils.ts # cn(), formatters
- └─ server/ # Server-only code (never imported by client)
 - └─ repositories/ # Data access layer
 - └─ family.repository.ts

- ├─ child.repository.ts
- ├─ transaction.repository.ts
- ├─ goal.repository.ts
- ├─ investment.repository.ts
- ├─ spending.repository.ts
- ├─ token-ledger.repository.ts
- ├─ audit.repository.ts
- ├─ base.repository.ts # Generic CRUD base
- ├─ services/ # Business logic layer
 - ├─ auth.service.ts
 - ├─ token.service.ts # INTERFACE ONLY (black-box)
 - ├─ goal.service.ts
 - ├─ transaction.service.ts
 - ├─ investment.service.ts
 - ├─ report.service.ts
 - ├─ notification.service.ts
 - ├─ upload.service.ts
 - ├─ audit.service.ts
 - ├─ scheduler.service.ts
- ├─ permissions/ # RBAC authorization
 - ├─ roles.ts # ADMIN, PARENT, CHILD
 - ├─ permissions.ts # Permission matrix
 - ├─ guards.ts # can(), requirePermission()
- ├─ jobs/ # Background job definitions
 - ├─ monthly-summary.job.ts
 - ├─ goal-reset.job.ts
 - ├─ streak-calc.job.ts
 - ├─ queue.ts # BullMQ queue setup
- ├─ emails/ # Email templates + sending
 - ├─ templates/
 - ├─ monthly-summary.tsx
 - ├─ goal-achieved.tsx
 - ├─ password-reset.tsx
 - ├─ sender.ts
- ├─ hooks/ # React hooks (client)
 - ├─ use-auth.ts
 - ├─ use-family.ts
 - ├─ use-goals.ts
 - ├─ use-notifications.ts
- ├─ stores/ # Zustand stores (UI state ONLY)
 - ├─ ui.store.ts # theme, sidebar, modals, filters
 - ├─ optimistic.store.ts # Optimistic update coordination
- ├─ types/ # Shared TypeScript types
 - ├─ api.ts # Request/response types
 - ├─ domain.ts # Domain entity types
 - ├─ events.ts # Domain event types
- ├─ middleware.ts # Auth + CSRF + rate-limit middleware
- ├─ public/ # Static assets
- ├─ .env.example # Environment variable template
- ├─ next.config.ts
- ├─ tailwind.config.ts
- ├─ tsconfig.json
- ├─ package.json
- ├─ README.md

Why each folder exists:

- **(auth) / (dashboard) / (child) route groups:** Isolates auth pages from the main app. Route groups don't affect URLs but enable distinct layouts.
- **features/:** Each feature is a self-contained module with its own components, server actions, Zod schemas, and types. No feature imports from another feature — they communicate via server actions + events.
- **server/:** Server-only code lives here. The `server/` prefix makes it visually obvious this code must never be imported by client components (Next.js will error if you try).
- **server/repositories/:** One repository per aggregate root. Repositories return domain entities, not Prisma rows. This isolates the ORM — if we switch from Prisma to Drizzle, only repositories change.
- **server/services/:** Services own business rules. TokenService is an interface only — the implementation is a black-box that must never be modified without domain-expert sign-off.
- **stores/:** UI state only. No domain data. The moment you put a `transactions` array in Zustand, you've broken the architecture — use React Query for server state.

3. Database Schema

Production Prisma schema targeting PostgreSQL with UUID primary keys, soft deletes, optimistic concurrency, and full indexing:

```
// prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// --- Enums -----

enum Role {
  ADMIN
  PARENT
  CHILD
}

enum TransactionType {
  SAVE
  WITHDRAW
  INVEST
  REDEEM
  PARENT_GIVE
}

enum GoalType {
  SAVE
  SPEND_LESS
}

enum GoalCadence {
  WEEKLY
  MONTHLY
  ANNUAL
}

enum GoalVisibility {
  PRIVATE
  REVEALED
}

enum InvestmentType {
  EQUITY
  BOND
  SAVINGS_BOND
  TREASURY_BILL
  UNIT_TRUST
}

enum InvestmentStatus {
  ACTIVE
  CLOSED
}
```

```

}

enum CurrencyCode {
    UGX
    USD
    KES
    TZS
    RWF
}

enum NotificationChannel {
    IN_APP
    EMAIL
    PUSH
}

enum NotificationStatus {
    UNREAD
    READ
    DISMISSED
}

enum AuditAction {
    LOGIN
    LOGOUT
    CREATE
    UPDATE
    DELETE
    TOKEN_AWARD
    TOKEN_REDEEM
    GOAL_CONTRIBUTE
    SETTINGS_CHANGE
    UPLOAD
}

// ---- Core Family Entities -----

model Family {
    id      String  @id @default(uuid())
    name    String
    currency CurrencyCode @default(UGX)
    annualTheme String @default("")
    monthlyQuote String @default("")
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
    deletedAt DateTime? // Soft delete

    users      User[]
    children   Child[]
    accounts   Account[]
    categories SpendingCategory[]
    settings   FamilySettings?

    @@index([deletedAt])
}

model User {
    id      String  @id @default(uuid())
    email   String  @unique
    passwordHash String?

```

```

name      String
role      Role   @default(PARENT)
familyId   String
emailVerified  DateTime?
image      String?
failedLoginAttempts Int @default(0)
lockedUntil  DateTime?
createdAt    DateTime @default(now())
updatedAt    DateTime @updatedAt
deletedAt    DateTime?

family      Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
sessions    Session[]
accounts    Account[] // For social login linking
auditLogs   AuditLog[]
notifications Notification[]
devices     Device[]
pushSubscriptions PushSubscription[]

@@index([familyId])
@@index([email])
@@index([deletedAt])
}

model Session {
  id      String @id @default(uuid())
  sessionToken String @unique
  userId   String
  expires   DateTime
  ipAddress String?
  userAgent String?
  createdAt DateTime @default(now())

  user      User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([expires])
}

// --- Family Members -----

model Child {
  id      String @id @default(uuid())
  familyId String
  name     String
  age      Int
  avatarColor String
  avatarPhoto String?
  currentAmount Int @default(0) // UGX live savings balance
  goalAmount  Int @default(0)
  goalName    String
  pinHash     String? // 4-digit PIN for child login
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  deletedAt   DateTime?
  version     Int @default(1) // Optimistic concurrency

  family      Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
  accounts    Account[]
  transactions Transaction[]

```

```

    spending    SpendingEntry[]
    investments Investment[]
    tokenLedger TokenLedgerEntry[]
    goals       Goal[] @relation("ChildGoals")

    @@index([familyId])
    @@index([deletedAt])
}

model ParentProfile {
  id          String @id @default(uuid())
  userId      String @unique // 1:1 with User (role=PARENT)
  familyId    String
  name        String
  role        String // "Mother" | "Father" | "Guardian"
  avatarColor String
  avatarPhoto String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  user        User @relation(fields: [userId], references: [id], onDelete: Cascade)
  family       Family @relation(fields: [familyId], references: [id])
  spending     SpendingEntry[]
  goals        Goal[] @relation("ParentGoals")

  @@index([familyId])
}

// --- Financial Entities -----

model Account {
  id          String @id @default(uuid())
  familyId    String
  childId     String?
  name        String // e.g. "Stanbic Junior"
  bankName    String?
  accountNumber String? // Last 4 digits only, encrypted
  balance     Int @default(0)
  currency    CurrencyCode @default(UGX)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  deletedAt   DateTime?
  version     Int @default(1)

  family       Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
  child        Child? @relation(fields: [childId], references: [id], onDelete: Cascade)
  transactions Transaction[]

  @@index([familyId])
  @@index([childId])
}

model Transaction {
  id          String @id @default(uuid())
  familyId    String
  childId     String
  type        TransactionType
  amount      Int @default(0)
  tokenDelta  Int @default(0)
  accountId   String?

```

```

investmentId String?
goalId    String? // Optional: link to a goal
note      String
timestamp DateTime @default(now())
createdAt DateTime @default(now())
version   Int      @default(1)

family    Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
child     Child  @relation(fields: [childId], references: [id], onDelete: Cascade)
account   Account? @relation(fields: [accountId], references: [id])
investment Investment? @relation(fields: [investmentId], references: [id])
goal      Goal?   @relation(fields: [goalId], references: [id])
auditLogs AuditLog[]

@@index([familyId, timestamp])
@@index([childId, timestamp])
@@index([type])
}

model SpendingCategory {
  id      String @id @default(uuid())
  familyId String
  name     String
  budget  Int    @default(0)
  createdAt DateTime @default(now())

  family    Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
  entries   SpendingEntry[]

  @@unique([familyId, name])
  @@index([familyId])
}

model SpendingEntry {
  id      String @id @default(uuid())
  familyId String
  ownerId  String // childId OR parentProfileId
  ownerKind String // "parent" | "child"
  ownerName String
  categoryId String?
  category String // Denormalized for historical records
  amount   Int    @default(0)
  note      String
  timestamp DateTime @default(now())
  createdAt DateTime @default(now())

  family    Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
  child     Child? @relation(fields: [childId], references: [id], onDelete: Cascade)
  parent    ParentProfile? @relation(fields: [parentId], references: [id], onDelete: Cascade)
  categoryRef SpendingCategory? @relation(fields: [categoryId], references: [id])
  childId   String?
  parentId  String?

  @@index([familyId, timestamp])
  @@index([ownerId, timestamp])
}

model Investment {
  id      String @id @default(uuid())
  familyId String

```

```

childId      String
name         String
type         InvestmentType
amountInvested Int @default(0)
currentValue Int @default(0)
status       InvestmentStatus @default(ACTIVE)
openedAt     DateTime @default(now())
closedAt     DateTime?
createdAt    DateTime @default(now())
updatedAt    DateTime @updatedAt
version      Int @default(1)

family       Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
child        Child @relation(fields: [childId], references: [id], onDelete: Cascade)
transactions Transaction[]

@@index([familyId, status])
@@index([childId, status])
}

model TokenLedgerEntry {
  id      String @id @default(uuid())
  familyId String
  childId String
  type    String // "parent_give" | "redeem"
  tokens  Int @default(0)
  note    String
  timestamp DateTime @default(now())
  createdAt DateTime @default(now())

  family       Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
  child        Child @relation(fields: [childId], references: [id], onDelete: Cascade)

  @@index([familyId, childId, timestamp])
  @@index([childId, type])
}

// --- Goals -----

model Goal {
  id      String @id @default(uuid())
  familyId String
  ownerId String
  ownerKind String // "parent" | "child"
  ownerName String
  title    String
  type     GoalType
  cadence  GoalCadence
  visibility GoalVisibility
  targetAmount Int @default(0)
  currentAmount Int @default(0)
  periodStart DateTime @default(now())
  note     String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?
  version  Int @default(1)

  family       Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
  child        Child? @relation("ChildGoals", fields: [childId], references: [id], onDelete: Cascade)

```

```

parent      ParentProfile? @relation("ParentGoals", fields: [parentId], references: [id], onDelete: Cascade)
childId     String?
parentId    String?
contributions GoalContribution[]
transactions Transaction[]

@@index([familyId, ownerId])
@@index([ownerId, visibility])
@@index([deletedAt])
}

model GoalContribution {
  id      String  @id @default(uuid())
  goalId  String
  amount  Int
  note    String?
  contributedBy String // userId or childId
  contributedAt DateTime @default(now())
  createdAt DateTime @default(now())

  goal    Goal    @relation(fields: [goalId], references: [id], onDelete: Cascade)

  @@index([goalId, contributedAt])
}

// --- Notifications -----

model Notification {
  id      String  @id @default(uuid())
  userId  String
  channel NotificationChannel
  status  NotificationStatus @default(UNREAD)
  title   String
  body    String
  metadata Json?
  createdAt DateTime @default(now())
  readAt   DateTime?

  user    User    @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId, status])
  @@index([createdAt])
}

// --- Audit Trail -----

model AuditLog {
  id      String  @id @default(uuid())
  userId  String?
  familyId String?
  action  AuditAction
  entityType String
  entityId String
  before  Json?
  after   Json?
  ipAddress String?
  userAgent String?
  createdAt DateTime @default(now())

  user    User?    @relation(fields: [userId], references: [id])

```

```

    @@index([userId, createdAt])
    @@index([familyId, createdAt])
    @@index([entityType, entityId])
    @@index([action])
}

// --- File Uploads -----

model FileUpload {
  id      String  @id @default(uuid())
  familyId String
  uploaderId String
  filename String
  mimeType String
  size     Int
  storageKey String // S3 key or Cloudinary public_id
  storageUrl String // CDN URL
  thumbnailUrl String?
  createdAt DateTime @default(now())
  deletedAt DateTime?

  @@index([familyId, createdAt])
  @@index([storageKey])
}

// --- Devices + Push -----

model Device {
  id      String  @id @default(uuid())
  userId   String
  name     String
  userAgent String?
  lastSeenAt DateTime @default(now())
  createdAt DateTime @default(now())

  user     User    @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
}

model PushSubscription {
  id      String  @id @default(uuid())
  userId   String
  endpoint String
  keys     Json
  createdAt DateTime @default(now())
  deactivatedAt DateTime?

  user     User    @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
}

// --- Email Log -----

model EmailLog {
  id      String  @id @default(uuid())
  to      String
  subject String

```



```

template String
status String // "sent" | "failed" | "bounced"
providerId String? // Resend message ID
error String?
createdAt DateTime @default(now())

@@index([to, createdAt])
@@index([status])
}

// ---- Family Settings (singleton per family) -----

model FamilySettings {
  id String @id @default(uuid())
  familyId String @unique
  notifyOnSave Boolean @default(true)
  notifyOnGoalReached Boolean @default(true)
  notifyOnTokenAward Boolean @default(true)
  monthlyReportDay Int @default(1) // Day of month to send report
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  family Family @relation(fields: [familyId], references: [id], onDelete: Cascade)
}

```

ERD Explanation

- **Family:** top-level tenant boundary. All other entities have familyId for row-level security + multi-tenant isolation.
- **User:** email + password (PARENT) or email + password (CHILD with limited permissions). Children log in with a 4-digit PIN instead of password — simpler for kids.
- **Transaction/Spending/Investment:** all financial entities are scoped to familyId. Indexes on [familyId, timestamp] enable fast per-family queries even at 50M+ rows.
- **Optimistic concurrency:** version field on Child, Account, Investment, Goal enables optimistic concurrency — prevents lost updates when two parents edit simultaneously.
- **Soft deletes:** deletedAt on Family, User, Child, Account, Goal, FileUpload. Queries filter WHERE deletedAt IS NULL. A cron job hard-deletes after 90 days.
- **GoalContribution:** GoalContribution tracks individual contributions to a goal (who, how much, when) — separate from the goal's currentAmount total.

Migration Strategy

- Use prisma migrate dev for local development. Each migration is a versioned SQL file checked into git.
- On deployment, prisma migrate deploy runs pending migrations in order. Never use db:push in production — it doesn't create migration files.
- For the SQLite-to-PostgreSQL transition (MVP → production), write a one-time data migration script that reads from SQLite and inserts into PostgreSQL via Prisma.
- For schema changes on large tables (e.g. adding a column to Transaction with 50M rows), use expand-then-contract: add the nullable column first, backfill in batches, then make it required in a follow-up migration.

Seed Strategy

- scripts/seed.ts creates 3 demo families with full data for local dev + staging.

- `scripts/seed-production.ts` creates only the ADMIN user + family on first deployment.
- Never seed production with demo data. Use a separate flag: `SEED_DEMO_DATA=true` only on local/staging.

4. Authentication Architecture

Auth.js (NextAuth v5) configuration with Credentials + Email providers, session management, and brute-force protection:

```
// src/lib/auth.ts
import NextAuth from "next-auth";
import Credentials from "next-auth/providers/credentials";
import Email from "next-auth/providers/email";
import { PrismaAdapter } from "@auth/prisma-adapter";
import bcrypt from "bcryptjs";
import { db } from "@lib/db";
import { rateLimit } from "@lib/rate-limit";
import { z } from "zod";

const credentialsSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
  rememberMe: z.boolean().optional(),
});

export const { handlers, auth, signIn, signOut } = NextAuth({
  adapter: PrismaAdapter(db),
  session: {
    strategy: "database", // DB-backed sessions for instant invalidation
    maxAge: 30 * 24 * 60 * 60, // 30 days
    updateAge: 24 * 60 * 60, // Update session every 24h
  },
  pages: {
    signIn: "/login",
    signOut: "/logout",
    error: "/login",
    verifyRequest: "/verify-request",
    newUser: "/register",
  },
  providers: [
    Credentials({
      name: "credentials",
      credentials: {
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" },
      },
      async authorize(credentials, request) {
        const parsed = credentialsSchema.safeParse(credentials);
        if (!parsed.success) return null;

        const { email, password } = parsed.data;

        // Rate limit: 5 attempts per IP per 15 minutes
        const ip = request.headers.get("x-forwarded-for") ?? "unknown";
        const { success } = await rateLimit(`login:${ip}`, 5, 900);
        if (!success) {
          throw new Error("Too many login attempts. Try again in 15 minutes.");
        }

        const user = await db.user.findUnique({
          where: { email: email.toLowerCase() },
        });
      },
    }),
  ],
});
```

```

if (!user || !user.passwordHash) return null;

// Check account lockout
if (user.lockedUntil && user.lockedUntil > new Date()) {
  throw new Error("Account locked. Try again later.");
}

// Verify password
const valid = await bcrypt.compare(password, user.passwordHash);
if (!valid) {
  // Increment failed attempts, lock after 5
  await db.user.update({
    where: { id: user.id },
    data: {
      failedLoginAttempts: { increment: 1 },
      lockedUntil:
        user.failedLoginAttempts + 1 >= 5
          ? new Date(Date.now() + 15 * 60 * 1000)
          : null,
    },
  });
  return null;
}

// Reset failed attempts on success
await db.user.update({
  where: { id: user.id },
  data: {
    failedLoginAttempts: 0,
    lockedUntil: null,
  },
});

return {
  id: user.id,
  email: user.email,
  name: user.name,
  role: user.role,
  familyId: user.familyId,
};
},
}),
Email({
  server: {
    host: process.env.SMTP_HOST,
    port: 587,
    auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS },
  },
  from: process.env.EMAIL_FROM,
}),
],
callbacks: {
  async jwt({ token, user }) {
    if (user) {
      token.role = user.role;
      token.familyId = user.familyId;
    }
    return token;
  },
  async session({ session, token }) {

```

```

    if (session.user) {
      session.user.id = token.sub!;
      session.user.role = token.role as Role;
      session.user.familyId = token.familyId as string;
    }
    return session;
  },
  authorized({ auth, request }) {
    // Protect all /dashboard and /api routes except /api/auth
    const pathname = request.nextUrl.pathname;
    if (pathname.startsWith("/dashboard") && !auth) return false;
    if (pathname.startsWith("/api/") && !pathname.startsWith("/api/auth") && !auth) {
      return false;
    }
    return true;
  },
},
events: {
  async signIn({ user, isNewUser }) {
    // Audit log
    await db.auditLog.create({
      data: {
        userId: user.id,
        action: "LOGIN",
        entityType: "User",
        entityId: user.id,
        ipAddress: "",
      },
    });
  },
},
});

```

Middleware (route protection + CSRF + rate limiting):

```

// src/middleware.ts
import { auth } from "@/lib/auth";
import { NextResponse } from "next/server";

export default auth((req) => {
  const { pathname } = req.nextUrl;
  const isLoggedIn = !!req.auth;

  // Public routes
  const publicRoutes = ["/", "/login", "/register", "/forgot-password",
    "/api/auth", "/api/health"];
  if (publicRoutes.some(r => pathname.startsWith(r))) {
    return NextResponse.next();
  }

  // Protected routes — require auth
  if (!isLoggedIn) {
    return NextResponse.redirect(new URL("/login", req.url));
  }

  // Role-based route protection
  const role = req.auth.user.role;
  if (pathname.startsWith("/dashboard") && role === "CHILD") {
    return NextResponse.redirect(new URL("/child", req.url));
  }
});

```

```

}
if (pathname.startsWith("/child") && role === "PARENT") {
  return NextResponse.redirect(new URL("/dashboard", req.url));
}

// Security headers
const response = NextResponse.next();
response.headers.set("X-Frame-Options", "DENY");
response.headers.set("X-Content-Type-Options", "nosniff");
response.headers.set("Referrer-Policy", "strict-origin-when-cross-origin");
response.headers.set("Permissions-Policy", "camera=(), microphone=(), geolocation=()");
response.headers.set("Strict-Transport-Security", "max-age=31536000; includeSubDomains");

return response;
});

export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon.ico).*)"],
};

```

Password Reset Flow:

```

// src/features/auth/actions.ts
"use server";

import { z } from "zod";
import bcrypt from "bcryptjs";
import { db } from "@lib/db";
import { sendPasswordResetEmail } from "@server/emails/sender";
import { generateToken } from "@lib/crypto";

const forgotPasswordSchema = z.object({
  email: z.string().email(),
});

export async function forgotPassword(formData: FormData) {
  const { email } = forgotPasswordSchema.parse({
    email: formData.get("email"),
  });

  const user = await db.user.findUnique({ where: { email } });
  if (!user) {
    // Don't reveal whether email exists — security best practice
    return { success: true };
  }

  const token = generateToken();
  const expires = new Date(Date.now() + 60 * 60 * 1000); // 1 hour

  await db.passwordResetToken.create({
    data: { userId: user.id, token, expires },
  });

  await sendPasswordResetEmail(email, token);

  return { success: true };
}

const resetPasswordSchema = z.object({

```

```

    token: z.string(),
    password: z.string().min(8).max(100),
  });

export async function resetPassword(formData: FormData) {
  const { token, password } = resetPasswordSchema.parse({
    token: formData.get("token"),
    password: formData.get("password"),
  });

  const resetToken = await db.passwordResetToken.findUnique({
    where: { token },
  });

  if (!resetToken || resetToken.expires < new Date()) {
    return { error: "Invalid or expired token" };
  }

  const passwordHash = await bcrypt.hash(password, 12);

  await db.$transaction([
    db.user.update({
      where: { id: resetToken.userId },
      data: { passwordHash },
    }),
    db.passwordResetToken.delete({ where: { id: resetToken.id } }),
    // Invalidate all existing sessions
    db.session.deleteMany({ where: { userId: resetToken.userId } }),
  ]);

  return { success: true };
}

```

RBAC Summary:

Role	Routes	Permissions
ADMIN	/admin/*	Manage all families, view system metrics, manage users
PARENT	/dashboard/*	Manage family, children, goals, investments, tokens, settings
CHILD	/child/*	View own dashboard, log spending, redeem tokens, view own goals

5. Authorization Architecture

Permission matrix + server-side guards:

```
// src/server/permissions/roles.ts
export const ROLES = {
  ADMIN: "ADMIN",
  PARENT: "PARENT",
  CHILD: "CHILD",
} as const;

export type Role = typeof ROLES[keyof typeof ROLES];

// src/server/permissions/permissions.ts
export const PERMISSIONS = {
  // Family
  FAMILY_VIEW: "family:view",
  FAMILY_MANAGE: "family:manage",

  // Children
  CHILD_VIEW: "child:view",
  CHILD_CREATE: "child:create",
  CHILD_UPDATE: "child:update",
  CHILD_DELETE: "child:delete",

  // Transactions
  TRANSACTION_VIEW: "transaction:view",
  TRANSACTION_CREATE: "transaction:create",
  TRANSACTION_DELETE: "transaction:delete",

  // Goals
  GOAL_VIEW: "goal:view",
  GOAL_CREATE: "goal:create",
  GOAL_UPDATE: "goal:update",
  GOAL_DELETE: "goal:delete",
  GOAL_CONTRIBUTE: "goal:contribute",

  // Investments
  INVESTMENT_VIEW: "investment:view",
  INVESTMENT_MANAGE: "investment:manage",

  // Tokens
  TOKEN_AWARD: "token:award",
  TOKEN_REDEEM: "token:redeem",
  TOKEN_VIEW_LEDGER: "token:view:ledger",

  // Reports
  REPORT_VIEW: "report:view",
  REPORT_GENERATE: "report:generate",

  // Settings
  SETTINGS_VIEW: "settings:view",
  SETTINGS_MANAGE: "settings:manage",

  // Uploads
  UPLOAD_AVATAR: "upload:avatar",
} as const;

export type Permission = typeof PERMISSIONS[keyof typeof PERMISSIONS];
```

```

// Role → Permission matrix
export const ROLE_PERMISSIONS: Record<Role, Permission[]> = {
  ADMIN: Object.values(PERMISSIONS), // All permissions

  PARENT: [
    PERMISSIONS.FAMILY_VIEW,
    PERMISSIONS.FAMILY_MANAGE,
    PERMISSIONS.CHILD_VIEW,
    PERMISSIONS.CHILD_CREATE,
    PERMISSIONS.CHILD_UPDATE,
    PERMISSIONS.CHILD_DELETE,
    PERMISSIONS.TRANSACTION_VIEW,
    PERMISSIONS.TRANSACTION_CREATE,
    PERMISSIONS.TRANSACTION_DELETE,
    PERMISSIONS.GOAL_VIEW,
    PERMISSIONS.GOAL_CREATE,
    PERMISSIONS.GOAL_UPDATE,
    PERMISSIONS.GOAL_DELETE,
    PERMISSIONS.GOAL_CONTRIBUTE,
    PERMISSIONS.INVESTMENT_VIEW,
    PERMISSIONS.INVESTMENT_MANAGE,
    PERMISSIONS.TOKEN_AWARD,
    PERMISSIONS.TOKEN_VIEW_LEDGER,
    PERMISSIONS.REPORT_VIEW,
    PERMISSIONS.REPORT_GENERATE,
    PERMISSIONS.SETTINGS_VIEW,
    PERMISSIONS.SETTINGS_MANAGE,
    PERMISSIONS.UPLOAD_AVATAR,
  ],

  CHILD: [
    PERMISSIONS.CHILD_VIEW, // Only own data
    PERMISSIONS.TRANSACTION_VIEW, // Only own transactions
    PERMISSIONS.TRANSACTION_CREATE, // Only save/redeem for self
    PERMISSIONS.GOAL_VIEW, // Only revealed + own private
    PERMISSIONS.GOAL_CONTRIBUTE, // Only to own goals
    PERMISSIONS.INVESTMENT_VIEW, // Only own investments
    PERMISSIONS.TOKEN_REDEEM, // Only own tokens
    PERMISSIONS.REPORT_VIEW, // Only own reports
    PERMISSIONS.UPLOAD_AVATAR, // Only own avatar
  ],
};

// src/server/permissions/guards.ts
import { PERMISSIONS, ROLE_PERMISSIONS, type Permission, type Role } from "../permissions";
import { auth } from "@lib/auth";
import { db } from "@lib/db";
import { AuthorizationError } from "@lib/errors";

export async function getCurrentUser() {
  const session = await auth();
  if (!session?.user) return null;
  return session.user;
}

export async function requireAuth() {
  const user = await getCurrentUser();
  if (!user) {
    throw new AuthorizationError("Authentication required");
  }
}

```

```

    return user;
  }

export async function requirePermission(permission: Permission) {
  const user = await requireAuth();
  const allowed = ROLE_PERMISSIONS[user.role as Role];
  if (!allowed.includes(permission)) {
    throw new AuthorizationError(`Missing permission: ${permission}`);
  }
  return user;
}

// Ensure the user can only access resources within their own family.
export async function requireFamilyAccess(familyId: string) {
  const user = await requireAuth();
  if (user.role === "ADMIN") return user;
  if (user.familyId !== familyId) {
    throw new AuthorizationError("Cross-family access denied");
  }
  return user;
}

// Ensure a child can only access their own data.
export async function requireChildAccess(childId: string) {
  const user = await requireAuth();

  if (user.role === "ADMIN") return user;

  if (user.role === "PARENT") {
    // Parent can access any child in their family
    const child = await db.child.findUnique({
      where: { id: childId },
      select: { familyId: true },
    });
    if (!child || child.familyId !== user.familyId) {
      throw new AuthorizationError("Child not found in your family");
    }
  }
  return user;
}

if (user.role === "CHILD") {
  // Child can only access their own record
  // (childId is stored on the User record for child users)
  if (user.childId !== childId) {
    throw new AuthorizationError("Cannot access another child's data");
  }
  return user;
}

throw new AuthorizationError("Invalid role");
}

// Ensure a child can only see revealed goals + their own private goals.
export async function filterGoalsByViewer<T extends { ownerId: string; visibility: string }>(
  goals: T[],
  viewerId: string,
  viewerRole: Role,
): Promise<T[]> {
  if (viewerRole === "ADMIN" || viewerRole === "PARENT") return goals;
  // CHILD: only revealed goals + their own private goals

```

```
return goals.filter(
  (g) => g.visibility === "REVEALED" || g.ownerId === viewerId,
);
}
```

Usage in Server Actions:

```
// src/features/goals/actions.ts
"use server";

import { z } from "zod";
import { requirePermission, requireFamilyAccess } from "@server/permissions/guards";
import { PERMISSIONS } from "@server/permissions/permissions";
import { GoalService } from "@server/services/goal.service";

const createGoalSchema = z.object({
  familyId: z.string().uuid(),
  ownerId: z.string().uuid(),
  ownerKind: z.enum(["parent", "child"]),
  title: z.string().min(1).max(80),
  type: z.enum(["save", "spend_less"]),
  cadence: z.enum(["weekly", "monthly", "annual"]),
  visibility: z.enum(["private", "revealed"]),
  targetAmount: z.number().int().positive(),
  note: z.string().max(120).optional(),
});

export async function createGoal(formData: FormData) {
  // 1. Authorize
  await requirePermission(PERMISSIONS.GOAL_CREATE);

  // 2. Validate
  const input = createGoalSchema.parse({
    familyId: formData.get("familyId"),
    ownerId: formData.get("ownerId"),
    // ... etc
  });

  // 3. Ensure family access
  await requireFamilyAccess(input.familyId);

  // 4. Delegate to service
  return GoalService.create(input);
}
```

6. API Architecture

REST API endpoints with Zod validation + typed responses:

Endpoint inventory:

Endpoint	Method	Purpose	Auth Required
/api/auth/*	GET/POST	NextAuth handlers	Public
/api/families	GET	List user's family	PARENT
/api/families	PATCH	Update family settings	PARENT
/api/children	GET	List children in family	PARENT
/api/children	POST	Create child	PARENT
/api/children/:id	GET	Get child detail	PARENT or CHILD(self)
/api/children/:id	PATCH	Update child	PARENT
/api/children/:id	DELETE	Soft-delete child	PARENT
/api/accounts	GET/POST	List/create accounts	PARENT
/api/transactions	GET	List transactions (filterable)	PARENT or CHILD(self)
/api/transactions	POST	Create transaction (save/withdraw/invest/redeem)	PARENT or CHILD(self)
/api/goals	GET/POST	List/create goals	PARENT or CHILD
/api/goals/:id	GET/PATCH/DELETE	Manage single goal	PARENT or CHILD(owner)
/api/goals/:id/contribute	POST	Contribute to goal	PARENT or CHILD
/api/investments	GET	List investments	PARENT or CHILD(self)
/api/tokens/award	POST	Award tokens (parent only)	PARENT
/api/tokens/redeem	POST	Redeem tokens	PARENT or CHILD(self)
/api/uploads/presign	POST	Get presigned upload URL	PARENT or CHILD
/api/uploads/confirm	POST	Confirm upload completed	PARENT or CHILD
/api/reports/monthly	GET	Generate monthly report	PARENT
/api/reports/export	GET	Export CSV/Excel	PARENT
/api/notifications	GET	List notifications	Any auth user
/api/notifications/:id/read	POST	Mark as read	Any auth user

Example: typed route handler with validation

```
// src/app/api/transactions/route.ts
import { NextRequest, NextResponse } from "next/server";
import { z } from "zod";
import { requirePermission, requireFamilyAccess, requireChildAccess } from "@server/permissions/guards";
import { PERMISSIONS } from "@server/permissions/permissions";
import { TransactionService } from "@server/services/transaction.service";
```

```

import { logger } from "@lib/logger";

const createTransactionSchema = z.object({
  familyId: z.string().uuid(),
  childId: z.string().uuid(),
  type: z.enum(["save", "withdraw", "invest", "redeem", "parent_give"]),
  amount: z.number().int().positive().max(1_000_000_000), // Max 1B UGX
  tokenDelta: z.number().int().min(0).default(0),
  accountId: z.string().uuid().optional(),
  investmentId: z.string().uuid().optional(),
  goalId: z.string().uuid().optional(),
  note: z.string().min(1).max(200),
});

export async function GET(req: NextRequest) {
  try {
    const user = await requirePermission(PERMISSIONS.TRANSACTION_VIEW);

    const { searchParams } = new URL(req.url);
    const familyId = searchParams.get("familyId");
    const childId = searchParams.get("childId");
    const type = searchParams.get("type");
    const limit = Math.min(Number(searchParams.get("limit") ?? "50"), 100);
    const offset = Number(searchParams.get("offset") ?? "0");

    if (familyId) await requireFamilyAccess(familyId);
    if (childId) await requireChildAccess(childId);

    const transactions = await TransactionService.list({
      familyId: familyId ?? user.familyId,
      childId: childId ?? undefined,
      type: type as any,
      limit,
      offset,
    });

    return NextResponse.json({ transactions, limit, offset });
  } catch (err) {
    logger.error({ err, path: req.url }, "GET /api/transactions failed");
    return NextResponse.json(
      { error: err instanceof Error ? err.message : "Internal error" },
      { status: err instanceof AuthorizationError ? 403 : 500 },
    );
  }
}

export async function POST(req: NextRequest) {
  try {
    const user = await requirePermission(PERMISSIONS.TRANSACTION_CREATE);

    const body = await req.json();
    const input = createTransactionSchema.parse(body);

    await requireFamilyAccess(input.familyId);
    await requireChildAccess(input.childId);

    const transaction = await TransactionService.create(input, user.id);

    return NextResponse.json({ transaction }, { status: 201 });
  } catch (err) {

```

```
if (err instanceof z.ZodError) {
  return NextResponse.json(
    { error: "Validation failed", details: err.flatten() },
    { status: 400 },
  );
}
logger.error({ err }, "POST /api/transactions failed");
return NextResponse.json(
  { error: err instanceof Error ? err.message : "Internal error" },
  { status: 500 },
);
}
}
```

7. State Management Architecture

Zustand is refactored to hold UI state ONLY. All domain data lives in React Query (server state cache).

What belongs in Zustand (UI state):

- theme (onyx, ivory, blush, red)
- sidebar open/closed
- active modal (saveMoney, giveTokens, etc.)
- filters (selected child, date range, spending category)
- optimistic update markers (pending mutations)

What does NOT belong in Zustand:

- children array, transactions array, goals array — use React Query
- currentAmount balances — server-derived, cached by React Query
- anything that comes from the database

```
// src/stores/ui.store.ts
"use client";

import { create } from "zustand";
import { persist } from "zustand/middleware";

type Theme = "onyx" | "ivory" | "blush" | "red";
type ModalName = "saveMoney" | "giveTokens" | "addSpending" | "redeemTokens" | "investmentDetail" |
"createGoal" | null;

interface UIState {
  // Theme
  theme: Theme;
  setTheme: (t: Theme) => void;

  // Sidebar
  sidebarOpen: boolean;
  toggleSidebar: () => void;
  setSidebarOpen: (open: boolean) => void;

  // Modals
  activeModal: ModalName;
  modalProps: Record<string, any>;
  openModal: (name: ModalName, props?: Record<string, any>) => void;
  closeModal: () => void;

  // Filters
  selectedChildId: string | null;
  setSelectedChild: (id: string | null) => void;
  dateRange: { start: Date; end: Date } | null;
  setDateRange: (range: { start: Date; end: Date } | null) => void;

  // Optimistic updates
  pendingMutations: Set<string>;
  addPendingMutation: (id: string) => void;
  removePendingMutation: (id: string) => void;
}
```



```

export const useUIStore = create<UIState>()(
  persist(
    (set, get) => ({
      theme: "onyx",
      setTheme: (theme) => set({ theme }),

      sidebarOpen: false,
      toggleSidebar: () => set((s) => ({ sidebarOpen: !s.sidebarOpen })),
      setSidebarOpen: (sidebarOpen) => set({ sidebarOpen }),

      activeModal: null,
      modalProps: {},
      openModal: (activeModal, modalProps = {}) => set({ activeModal, modalProps }),
      closeModal: () => set({ activeModal: null, modalProps: {} }),

      selectedChildId: null,
      setSelectedChild: (selectedChildId) => set({ selectedChildId }),
      dateRange: null,
      setDateRange: (dateRange) => set({ dateRange }),

      pendingMutations: new Set(),
      addPendingMutation: (id) =>
        set((s) => {
          const next = new Set(s.pendingMutations);
          next.add(id);
          return { pendingMutations: next };
        }),
      removePendingMutation: (id) =>
        set((s) => {
          const next = new Set(s.pendingMutations);
          next.delete(id);
          return { pendingMutations: next };
        }),
    }),
    {
      name: "planned-ui",
      partialize: (state) => ({ theme: state.theme }), // Only persist theme
    },
  ),
);

```

Server state with React Query:

```

// src/hooks/use-transactions.ts
"use client";

import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { useUIStore } from "@stores/ui.store";

export function useTransactions(childId?: string) {
  const selectedChildId = useUIStore((s) => s.selectedChildId);
  const effectiveChildId = childId ?? selectedChildId;

  return useQuery({
    queryKey: ["transactions", effectiveChildId],
    queryFn: async () => {
      const params = new URLSearchParams();
      if (effectiveChildId) params.set("childId", effectiveChildId);
      const res = await fetch(`/api/transactions?${params}`);
    },
  });
}

```

```

    if (!res.ok) throw new Error("Failed to load transactions");
    return res.json();
  },
  staleTime: 30_000, // 30 seconds
});
}

export function useCreateTransaction() {
  const queryClient = useQueryClient();
  const addPending = useUIStore((s) => s.addPendingMutation);
  const removePending = useUIStore((s) => s.removePendingMutation);

  return useMutation({
    mutationFn: async (input: CreateTransactionInput) => {
      const res = await fetch("/api/transactions", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(input),
      });
      if (!res.ok) throw new Error("Failed to create transaction");
      return res.json();
    },
    onMutate: async (input) => {
      // Optimistic update
      const tempId = `temp-${Date.now()}`;
      addPending(tempId);

      await queryClient.cancelQueries({ queryKey: ["transactions", input.childId] });

      const previous = queryClient.getQueryData(["transactions", input.childId]);

      queryClient.setQueryData(["transactions", input.childId], (old: any) => ({
        ...old,
        transactions: [
          {
            id: tempId,
            ...input,
            timestamp: new Date().toISOString(),
            pending: true,
          },
          ...(old?.transactions ?? []),
        ],
      }));

      return { previous, tempId };
    },
    onError: (err, input, context) => {
      // Roll back on error
      queryClient.setQueryData(["transactions", input.childId], context?.previous);
    },
    onSuccess: (data, input, context) => {
      removePending(context!.tempId);
      // Replace temp with real
      queryClient.setQueryData(["transactions", input.childId], (old: any) => ({
        ...old,
        transactions: old.transactions.map((t: any) =>
          t.id === context!.tempId ? data.transaction : t,
        ),
      }));
    },
  });
}

```

```
onSettled: (data, error, input) => {  
  queryClient.invalidateQueries({ queryKey: ["transactions", input.childId] });  
  queryClient.invalidateQueries({ queryKey: ["children"] }); // Balance changed  
},  
});  
}
```

8. Services Layer

Services own business rules. Each service is a stateless module that receives input, calls repositories, and returns results. The TokenService is an INTERFACE ONLY — never modify token calculations.

```
// src/server/services/token.service.ts
//
=====
// TOKEN SERVICE — BLACK-BOX INTERFACE
//
=====
// DO NOT modify token pricing, incentive calculations, conversion formulas,
// or reward algorithms. These are proprietary domain rules.
// This interface is the ONLY public API for token operations.
//
=====
=====

export interface TokenService {
  /**
   * Parent awards tokens to a child.
   * The parent's account is charged at TOKEN_BUY_RATE per token.
   * Returns the transaction record.
   */
  award(input: {
    familyId: string;
    childId: string;
    tokens: number;
    note: string;
    awardedBy: string;
  }): Promise<TokenAwardResult>;

  /**
   * Child redeems tokens for cash (credited to savings).
   * Uses TOKEN_REDEEM_RATE per token.
   * Returns the transaction record + cash value.
   */
  redeem(input: {
    familyId: string;
    childId: string;
    tokens: number;
  }): Promise<TokenRedeemResult>;

  /**
   * Returns the current balance for a child.
   * Balance = awarded tokens - redeemed tokens.
   */
  getBalance(childId: string): Promise<number>;

  /**
   * Returns the cost to the parent for purchasing N tokens.
   * Uses TOKEN_BUY_RATE. This is a pure calculation — no side effects.
   */
  calculatePurchaseCost(tokens: number): number;
}
```

```

/**
 * Returns the cash value a child receives for redeeming N tokens.
 * Uses TOKEN_REDEEM_RATE. This is a pure calculation — no side effects.
 */
calculateRedeemValue(tokens: number): number;
}

export interface TokenAwardResult {
  transactionId: string;
  ledgerEntryId: string;
  tokensAwarded: number;
  parentCost: number; // UGX
}

export interface TokenRedeemResult {
  transactionId: string;
  ledgerEntryId: string;
  tokensRedeemed: number;
  cashValue: number; // UGX credited to savings
  newBalance: number;
}

//
=====
=====
// IMPLEMENTATION — lives in a separate file, injected via DI
// src/server/services/token.service.impl.ts
// (Contents are proprietary — do not read or modify without authorization)
//
=====
=====

// src/server/services/goal.service.ts
import { GoalRepository } from "@server/repositories/goal.repository";
import { AuditService } from "../audit.service";
import { NotificationService } from "../notification.service";
import { logger } from "@lib/logger";

export const GoalService = {
  async create(input: CreateGoalInput, userId: string) {
    const goal = await GoalRepository.create(input);

    await AuditService.log({
      userId,
      action: "CREATE",
      entityType: "Goal",
      entityId: goal.id,
      after: goal,
    });

    logger.info({ goalId: goal.id, userId }, "Goal created");
    return goal;
  },

  async contribute(goalId: string, amount: number, contributedBy: string) {
    const goal = await GoalRepository.findById(goalId);
    if (!goal) throw new NotFoundError("Goal not found");

    // Optimistic concurrency
    const updated = await GoalRepository.updateWithVersion(

```

```

    goal.id,
    goal.version,
    {
      currentAmount: goal.currentAmount + amount,
    },
  );

  await GoalRepository.addContribution({
    goalId,
    amount,
    contributedBy,
  });

  await AuditService.log({
    userId: contributedBy,
    action: "GOAL_CONTRIBUTE",
    entityType: "Goal",
    entityId: goalId,
    after: { currentAmount: updated.currentAmount },
  });

  // Check if goal reached
  if (updated.currentAmount >= updated.targetAmount && goal.currentAmount < goal.targetAmount) {
    await NotificationService.notifyGoalReached(updated);
  }

  return updated;
},

async delete(goalId: string, userId: string) {
  const goal = await GoalRepository.findById(goalId);
  if (!goal) throw new NotFoundError("Goal not found");

  await AuditService.log({
    userId,
    action: "DELETE",
    entityType: "Goal",
    entityId: goalId,
    before: goal,
  });

  await GoalRepository.softDelete(goalId);
},
};

```

9. Repository Layer

Repositories own data access. They return domain entities, not Prisma rows. This isolates the ORM

— if we switch from Prisma to Drizzle, only repositories change.

```
// src/server/repositories/base.repository.ts
import { db } from "@lib/db";

export interface BaseRepository<T, CreateInput, UpdateInput> {
  findById(id: string): Promise<T | null>;
  findMany(filter: Record<string, any>, opts?: { limit?: number; offset?: number }): Promise<T[]>;
  create(input: CreateInput): Promise<T>;
  update(id: string, input: UpdateInput): Promise<T>;
  updateWithVersion(id: string, version: number, input: Partial<UpdateInput>): Promise<T>;
  softDelete(id: string): Promise<void>;
}

// src/server/repositories/goal.repository.ts
import { db } from "@lib/db";
import type { Goal } from "@types/domain";

export const GoalRepository = {
  async findById(id: string): Promise<Goal | null> {
    const row = await db.goal.findUnique({
      where: { id, deletedAt: null },
      include: { contributions: true },
    });
    return row ? toDomain(row) : null;
  },

  async findMany(filter: {
    familyId?: string;
    ownerId?: string;
    visibility?: string;
  }, opts?: { limit?: number; offset?: number }): Promise<Goal[]> {
    const rows = await db.goal.findMany({
      where: {
        ...filter,
        deletedAt: null,
      },
      orderBy: { createdAt: "desc" },
      take: opts?.limit ?? 50,
      skip: opts?.offset ?? 0,
    });
    return rows.map(toDomain);
  },

  async create(input: CreateGoalInput): Promise<Goal> {
    const row = await db.goal.create({
      data: {
        ...input,
        periodStart: new Date(),
        version: 1,
      },
    });
    return toDomain(row);
  },

  async updateWithVersion(id: string, version: number, patch: Partial<Goal>): Promise<Goal> {
```

```

try {
  const row = await db.goal.update({
    where: { id, version }, // Optimistic concurrency
    data: {
      ...patch,
      version: { increment: 1 },
    },
  });
  return toDomain(row);
} catch (err) {
  if (err.code === "P2025") {
    throw new ConcurrencyError("Goal was modified by another user. Refresh and try again.");
  }
  throw err;
}
},

async addContribution(input: {
  goalId: string;
  amount: number;
  contributedBy: string;
}): Promise<void> {
  await db.goalContribution.create({ data: input });
},

async softDelete(id: string): Promise<void> {
  await db.goal.update({
    where: { id },
    data: { deletedAt: new Date() },
  });
},
};

function toDomain(row: any): Goal {
  return {
    id: row.id,
    familyId: row.familyId,
    ownerId: row.ownerId,
    ownerKind: row.ownerKind,
    ownerName: row.ownerName,
    title: row.title,
    type: row.type,
    cadence: row.cadence,
    visibility: row.visibility,
    targetAmount: row.targetAmount,
    currentAmount: row.currentAmount,
    periodStart: row.periodStart.getTime(),
    note: row.note ?? undefined,
    createdAt: row.createdAt.getTime(),
    version: row.version,
  };
}

```


10. Notifications System

Multi-channel notifications (in-app, email, push) with preferences + retry logic.

```
// src/server/services/notification.service.ts
import { db } from "@lib/db";
import { emailSender } from "@server/emails/sender";
import { pushSender } from "@server/notifications/push-sender";
import { logger } from "@lib/logger";

type NotificationEvent =
  | { type: "GOAL_REACHED"; goalId: string; ownerId: string }
  | { type: "SAVINGS_MILESTONE"; childId: string; amount: number }
  | { type: "TOKEN_AWARDED"; childId: string; tokens: number; awardedBy: string }
  | { type: "MONTHLY_SUMMARY"; familyId: string }
  | { type: "INVESTMENT_UPDATE"; childId: string; investmentId: string };

export const NotificationService = {
  async notify(event: NotificationEvent) {
    const recipients = await this.resolveRecipients(event);

    for (const recipient of recipients) {
      const prefs = await this.getPreferences(recipient.id);

      // Always create in-app notification
      await db.notification.create({
        data: {
          userId: recipient.id,
          channel: "IN_APP",
          title: this.getTitle(event),
          body: this.getBody(event),
          metadata: event as any,
        },
      });

      // Send email if enabled
      if (prefs.email && this.shouldEmail(event)) {
        await this.queueEmail(recipient, event);
      }

      // Send push if enabled
      if (prefs.push) {
        await this.queuePush(recipient, event);
      }
    }
  },

  async notifyGoalReached(goal: any) {
    await this.notify({ type: "GOAL_REACHED", goalId: goal.id, ownerId: goal.ownerId });
  },

  async notifyTokenAwarded(childId: string, tokens: number, awardedBy: string) {
    await this.notify({ type: "TOKEN_AWARDED", childId, tokens, awardedBy });
  },

  private async resolveRecipients(event: NotificationEvent): Promise<User[]> {
    switch (event.type) {
      case "GOAL_REACHED":
        // Notify goal owner + all parents in family
        const goal = await db.goal.findUnique({ where: { id: event.goalId } });

```

```

    const parents = await db.user.findMany({
      where: { familyId: goal.familyId, role: "PARENT" },
    });
    return parents;

    case "TOKEN_AWARDED":
      // Notify the child + their parents
      const child = await db.child.findUnique({ where: { id: event.childId } });
      return db.user.findMany({
        where: { familyId: child.familyId, role: "PARENT" },
      });

    case "MONTHLY_SUMMARY":
      return db.user.findMany({
        where: { familyId: event.familyId, role: "PARENT" },
      });
  }
},

private shouldEmail(event: NotificationEvent): boolean {
  // Only email for significant events, not every token award
  return ["GOAL_REACHED", "MONTHLY_SUMMARY"].includes(event.type);
},
};

```

11. Scheduler System

Vercel Cron triggers scheduled jobs. Heavy processing runs on BullMQ + Redis.

```

// vercel.json
{
  "crons": [
    { "path": "/api/cron/monthly-summary", "schedule": "0 8 1 * *" },
    { "path": "/api/cron/goal-reset", "schedule": "0 0 * * 0" },
    { "path": "/api/cron/streak-calc", "schedule": "0 1 * * *" },
    { "path": "/api/cron/allowance-payout", "schedule": "0 9 * * 1" }
  ]
}

// src/app/api/cron/monthly-summary/route.ts
import { NextRequest, NextResponse } from "next/server";
import { MonthlySummaryJob } from "@server/jobs/monthly-summary.job";

export async function GET(req: NextRequest) {
  // Verify Vercel Cron signature
  const authHeader = req.headers.get("authorization");
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  try {
    await MonthlySummaryJob.run();
    return NextResponse.json({ ok: true });
  } catch (err) {
    console.error("Monthly summary job failed:", err);
    return NextResponse.json({ error: "Job failed" }, { status: 500 });
  }
}

```

```
// src/server/jobs/monthly-summary.job.ts
import { db } from "@lib/db";
import { NotificationService } from "@server/services/notification.service";
import { ReportService } from "@server/services/report.service";

export const MonthlySummaryJob = {
  async run() {
    const families = await db.family.findMany({ where: { deletedAt: null } });

    for (const family of families) {
      try {
        const report = await ReportService.generateMonthlySummary(family.id);

        await NotificationService.notify({
          type: "MONTHLY_SUMMARY",
          familyId: family.id,
        });

        // Idempotency: mark as sent
        await db.familySettings.update({
          where: { familyId: family.id },
          data: { lastMonthlySummaryAt: new Date() },
        });
      } catch (err) {
        console.error(`Summary for family ${family.id} failed:`, err);
        // Continue to next family — one failure shouldn't block others
      }
    }
  },
};
```

12. Reporting System

CSV/PDF exports + monthly reports via background jobs.

```
// src/server/services/report.service.ts
import { db } from "@lib/db";
import { formatCurrency } from "@lib/currency";
import { emailSender } from "@server/emails/sender";

export const ReportService = {
  async generateMonthlySummary(familyId: string) {
    const now = new Date();
    const monthStart = new Date(now.getFullYear(), now.getMonth() - 1, 1);
    const monthEnd = new Date(now.getFullYear(), now.getMonth(), 0, 23, 59, 59);

    const family = await db.family.findUnique({ where: { id: familyId } });
    const children = await db.child.findMany({ where: { familyId, deletedAt: null } });

    const summary = {
      familyName: family.name,
      month: monthStart.toLocaleDateString("en-US", { month: "long", year: "numeric" }),
      currency: family.currency,
      children: [] as any[],
      totalSaved: 0,
      totalSpent: 0,
      goalsReached: 0,
    };

    for (const child of children) {
      const saved = await db.transaction.aggregate({
        where: {
          childId: child.id,
          type: "SAVE",
          timestamp: { gte: monthStart, lte: monthEnd },
        },
        _sum: { amount: true },
      });

      const spent = await db.spendingEntry.aggregate({
        where: {
          ownerId: child.id,
          timestamp: { gte: monthStart, lte: monthEnd },
        },
        _sum: { amount: true },
      });

      const goalsReached = await db.goal.count({
        where: {
          ownerId: child.id,
          currentAmount: { gte: db.goal.fields.targetAmount },
        },
      });

      summary.children.push({
        name: child.name,
        saved: saved._sum.amount ?? 0,
        spent: spent._sum.amount ?? 0,
        goalsReached,
      });
      summary.totalSaved += saved._sum.amount ?? 0;
    }
  }
};
```

```

        summary.totalSpent += spent._sum.amount ?? 0;
        summary.goalsReached += goalsReached;
    }

    return summary;
},

async exportTransactionsCSV(familyId: string, opts: { childId?: string; from?: Date; to?: Date }) {
    const transactions = await db.transaction.findMany({
        where: {
            familyId,
            childId: opts.childId,
            timestamp: { gte: opts.from, lte: opts.to },
        },
        orderBy: { timestamp: "desc" },
    });

    const headers = ["Date", "Child", "Type", "Amount (UGX)", "Tokens", "Note"];
    const rows = transactions.map((t) => [
        new Date(t.timestamp).toISOString(),
        t.child.name,
        t.type,
        t.amount.toString(),
        t.tokenDelta.toString(),
        t.note,
    ]);

    return [headers, ...rows].map((r) => r.join(",")).join("\n");
},

async emailMonthlyReport(familyId: string) {
    const summary = await this.generateMonthlySummary(familyId);
    const parents = await db.user.findMany({
        where: { familyId, role: "PARENT" },
    });

    for (const parent of parents) {
        await emailSender.send({
            to: parent.email,
            subject: `${summary.month} Summary for ${summary.familyName}`,
            template: "monthly-summary",
            data: summary,
        });
    }
},
};

```

13. File Upload System

Presigned S3 uploads + image processing via Cloudinary.

```
// src/app/api/uploads/presign/route.ts
import { NextRequest, NextResponse } from "next/server";
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";
import { getSignedUrl } from "@aws-sdk/s3-request-presigner";
import { requireAuth } from "@server/permissions/guards";
import { z } from "zod";

const s3 = new S3Client({
  region: process.env.S3_REGION,
  credentials: {
    accessKeyId: process.env.S3_ACCESS_KEY!,
    secretAccessKey: process.env.S3_SECRET_KEY!,
  },
});

const presignSchema = z.object({
  filename: z.string(),
  mimeType: z.string().regex(/^image\/(jpeg|png|webp)$/),
  size: z.number().int().positive().max(5_000_000), // 5MB max
  purpose: z.enum(["avatar", "document"]),
});

export async function POST(req: NextRequest) {
  const user = await requireAuth();

  const body = await req.json();
  const input = presignSchema.parse(body);

  const key = `uploads/${user.familyId}/${input.purpose}/${crypto.randomUUID()}-${input.filename}`;

  const command = new PutObjectCommand({
    Bucket: process.env.S3_BUCKET!,
    Key: key,
    ContentType: input.mimeType,
    ContentLength: input.size,
  });

  const uploadUrl = await getSignedUrl(s3, command, { expiresIn: 300 });

  return NextResponse.json({ uploadUrl, key });
}

// src/app/api/uploads/confirm/route.ts
export async function POST(req: NextRequest) {
  const user = await requireAuth();

  const { key, purpose } = await req.json();

  // For avatars: generate thumbnail via Cloudinary
  let thumbnailUrl: string | undefined;
  if (purpose === "avatar") {
    thumbnailUrl = await generateThumbnail(key, 64);
  }

  // Record in database
  const upload = await db.fileUpload.create({
```

```
data: {
  familyId: user.familyId,
  uploaderId: user.id,
  filename: key.split("/").pop(),
  mimeType: "image/jpeg",
  size: 0, // Updated via S3 event
  storageKey: key,
  storageUrl: `${process.env.CDN_BASE_URL}/${key}`,
  thumbnailUrl,
},
});

return NextResponse.json({ upload });
}
```

14. Audit Trail System

Immutable audit log capturing who/what/when/where/before/after for every state change.

```
// src/server/services/audit.service.ts
import { db } from "@lib/db";
import { auth } from "@lib/auth";
import { headers } from "next/headers";

export const AuditService = {
  async log(input: {
    userId?: string;
    action: string;
    entityType: string;
    entityId: string;
    before?: any;
    after?: any;
  }) {
    const session = await auth();
    const headerList = headers();

    await db.auditLog.create({
      data: {
        userId: input.userId ?? session?.user?.id,
        familyId: session?.user?.familyId,
        action: input.action,
        entityType: input.entityType,
        entityId: input.entityId,
        before: input.before ?? null,
        after: input.after ?? null,
        ipAddress: headerList.get("x-forwarded-for") ?? null,
        userAgent: headerList.get("user-agent") ?? null,
      },
    });
  },

  async query(filter: {
    userId?: string;
    familyId?: string;
    action?: string;
    entityType?: string;
    entityId?: string;
    from?: Date;
    to?: Date;
    limit?: number;
    offset?: number;
  }) {
    return db.auditLog.findMany({
      where: {
        userId: filter.userId,
        familyId: filter.familyId,
        action: filter.action as any,
        entityType: filter.entityType,
        entityId: filter.entityId,
        createdAt: { gte: filter.from, lte: filter.to },
      },
      orderBy: { createdAt: "desc" },
      take: filter.limit ?? 50,
      skip: filter.offset ?? 0,
    });
  }
};
```



```
    },  
  
    async exportCSV(filter: Parameters<typeof this.query>[0]): Promise<string> {  
      const logs = await this.query({ ...filter, limit: 10000 });  
      const headers = ["Timestamp", "User", "Action", "Entity", "Entity ID", "IP", "Details"];  
      const rows = logs.map(l => [  
        l.createdAt.toISOString(),  
        l.userId ?? "",  
        l.action,  
        l.entityType,  
        l.entityId,  
        l.ipAddress ?? "",  
        JSON.stringify({ before: l.before, after: l.after }),  
      ]);  
      return [headers, ...rows].map((r) => r.map((c) => `${c}`).join(",")).join("\n");  
    },  
  },  
};
```

15. Security Architecture

Security checklist + implementation:

Layer	Threat	Mitigation
Transport	MITM	HTTPS-only, HSTS header (max-age 1 year)
Auth	Brute force	Rate limit 5 logins/15min per IP, lockout after 5 failures
Auth	Credential stuffing	bcrypt cost 12, password breach check via HaveIBeenPwned API
Session	Session hijacking	HttpOnly + Secure + SameSite=Lax cookies, rotate session token on login
Session	CSRF	SameSite cookies + double-submit token for mutations
Input	SQL injection	Prisma parameterized queries (never raw SQL)
Input	XSS	React auto-escaping + DOMPurify for user HTML
Input	NoSQL injection	Zod validation on every API + server action input
Authz	IDOR	requireFamilyAccess + requireChildAccess guards on every route
Authz	Privilege escalation	Role checks via requirePermission before every mutation
Files	Malicious upload	MIME type validation + file size limit + virus scan (ClamAV)
Rate	API abuse	Redis-backed rate limiter: 100 req/min per user, 1000/min per IP
Secrets	Leaked keys	Environment validation via zod, no secrets in git
Deps	Vulnerable packages	Dependabot + npm audit in CI
Headers	Clickjacking	X-Frame-Options: DENY
Headers	MIME sniffing	X-Content-Type-Options: nosniff

```
// src/lib/rate-limit.ts
import { Redis } from "@upstash/redis";

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_URL!,
  token: process.env.UPSTASH_REDIS_TOKEN!,
});

export async function rateLimit(
```

```

    key: string,
    limit: number,
    windowSeconds: number,
  ): Promise<{ success: boolean; remaining: number }> {
    const count = await redis.incr(key);
    if (count === 1) {
      await redis.expire(key, windowSeconds);
    }
    return {
      success: count <= limit,
      remaining: Math.max(0, limit - count),
    };
  }
}

```

```

// src/lib/env.ts
import { z } from "zod";

```

```

const envSchema = z.object({
  DATABASE_URL: z.string().url(),
  NEXTAUTH_SECRET: z.string().min(32),
  NEXTAUTH_URL: z.string().url(),
  SMTP_HOST: z.string(),
  SMTP_USER: z.string(),
  SMTP_PASS: z.string(),
  EMAIL_FROM: z.string().email(),
  S3_REGION: z.string(),
  S3_ACCESS_KEY: z.string(),
  S3_SECRET_KEY: z.string(),
  S3_BUCKET: z.string(),
  CDN_BASE_URL: z.string().url(),
  UPSTASH_REDIS_URL: z.string().url(),
  UPSTASH_REDIS_TOKEN: z.string(),
  CRON_SECRET: z.string().min(32),
  SENTRY_DSN: z.string().optional(),
});

```

```

export const env = envSchema.parse(process.env);

```

16. Deployment Architecture

Vercel + PostgreSQL (Neon/Supabase) + S3 + Upstash Redis + Resend Email.

```
# .env.example

# Database
DATABASE_URL="postgresql://user:pass@host:5432/planned?schema=public"

# Auth
NEXTAUTH_SECRET="generate-with-openssl-rand-base64-32"
NEXTAUTH_URL="https://planned.app"

# Email (Resend)
RESEND_API_KEY="re_XXXXX"
EMAIL_FROM="Planned <hello@planned.app>"

# File Storage (S3 / Cloudinary)
S3_REGION="us-east-1"
S3_ACCESS_KEY="AKIAXXXXX"
S3_SECRET_KEY="XXXXX"
S3_BUCKET="planned-uploads"
CDN_BASE_URL="https://cdn.planned.app"

# Cache + Queue (Upstash Redis)
UPSTASH_REDIS_URL="https://xxx.upstash.io"
UPSTASH_REDIS_TOKEN="XXXXX"

# Cron
CRON_SECRET="generate-with-openssl-rand-base64-32"

# Observability
SENTRY_DSN="https://xxx@sentry.io/xxx"

# Feature flags
NEXT_PUBLIC_APP_URL="https://planned.app"
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_test_XXX" # Future monetization
```

CI/CD Pipeline (GitHub Actions):

```
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: oven-sh/setup-bun@v1
      - run: bun install
      - run: bun run lint
      - run: bun run typecheck
      - run: bun run test
```

```
- run: bun run prisma generate
- run: bun run prisma migrate deploy
env:
  DATABASE_URL: \${{ secrets.DATABASE_URL }}
```

```
deploy:
  needs: test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: amondnet/vercel-action@v25
      with:
        vercel-token: \${{ secrets.VERCEL_TOKEN }}
        vercel-org-id: \${{ secrets.VERCEL_ORG_ID }}
        vercel-project-id: \${{ secrets.VERCEL_PROJECT_ID }}
        vercel-args: '--prod'
```

17. Environment Variables

Variable	Purpose	Required
DATABASE_URL	PostgreSQL connection string	Yes
NEXTAUTH_SECRET	JWT + session encryption (32+ chars)	Yes
NEXTAUTH_URL	App URL for callbacks	Yes
RESEND_API_KEY	Email sending	Yes
EMAIL_FROM	From address	Yes
S3_REGION / S3_ACCESS_KEY / S3_SECRET_KEY / S3_BUCKET	File storage	Yes
CDN_BASE_URL	CloudFront/Cloudinary URL	Yes
UPSTASH_REDIS_URL / UPSTASH_REDIS_TOKEN	Cache + rate limit + queue	Yes
CRON_SECRET	Protects cron endpoints	Yes
SENTRY_DSN	Error tracking	Optional
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Future payments	Optional

18. Migration Plan

Phase 1: SQLite → PostgreSQL (1 day)

- Update DATABASE_URL to PostgreSQL connection string
- Run prisma migrate dev to create migration from schema
- Write one-time data migration script: read from SQLite, insert into PostgreSQL
- Run prisma migrate deploy on production

Phase 2: Add authentication (3 days)

- Configure NextAuth with Credentials + Email providers
- Add middleware for route protection
- Migrate existing Family data to have a User (parent) account
- Add login/register/forgot-password pages

Phase 3: Multi-tenant isolation (2 days)

- Add familyId to all existing entities (they already have it)
- Add requireFamilyAccess guard to every route + server action
- Verify no cross-family data leakage with integration tests

Phase 4: File uploads (2 days)

- Set up S3 bucket + CloudFront distribution
- Build presigned upload API
- Migrate avatar photos from base64 (in DB) to S3 URLs

Phase 5: Notifications + Scheduler (3 days)

- Set up Resend for email
- Build notification service + in-app dropdown
- Configure Vercel Cron for monthly summaries + goal resets

Phase 6: Audit + Reports (2 days)

- Add audit log to every mutation
- Build CSV export endpoint
- Build monthly PDF report generator

19. Scaling Strategy

Target: 10,000 families, 50,000 children, millions of transactions.

Bottleneck	Solution
Database reads	Read replicas + React Query caching (30s staleTime)
Database writes	Connection pooling via PgBouncer, optimistic concurrency
Large tables (Transaction)	Partition by year + familyId. Archive rows > 2 years to cold storage.
File storage	S3 + CloudFront CDN. No files in database.
Background jobs	BullMQ + Redis. Vercel Cron triggers, workers process.
Search	PostgreSQL full-text search initially. Migrate to Meilisearch if needed.
Rate limiting	Redis-backed sliding window. 100 req/min/user.
Memory	Vercel serverless functions: 1GB max. Stream large responses.

Partitioning strategy for Transaction table (50M+ rows):

```
-- PostgreSQL native partitioning by year + familyId hash
CREATE TABLE transaction (
  id UUID,
  family_id UUID,
  child_id UUID,
  type TEXT,
  amount INTEGER,
  timestamp TIMESTAMPTZ,
  ...
) PARTITION BY RANGE (timestamp);

CREATE TABLE transaction_2025 PARTITION OF transaction
FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');

CREATE TABLE transaction_2026 PARTITION OF transaction
FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');

-- Sub-partition by familyId hash for write scaling
CREATE TABLE transaction_2025_f0 PARTITION OF transaction_2025
FOR VALUES WITH (modulus 4, remainder 0);
-- ... etc
```

20. Implementation Roadmap

12-week roadmap to production:

Week	Focus	Deliverables
1-2	Database + Auth	PostgreSQL migration, NextAuth, login/register, RBAC guards
3	Multi-tenant isolation	familyId on all entities, guards on all

		routes
4	Repository + Service layers	Refactor all DB access behind repositories, extract services
5	Token economy (black-box)	TokenService interface, wire up award/redeem via interface
6	File uploads	S3 presigned uploads, avatar migration from base64
7	Notifications	In-app + email, notification preferences, Resend integration
8	Scheduler + Jobs	Vercel Cron, monthly summaries, goal resets, streak calc
9	Reporting	CSV export, PDF monthly report, email delivery
10	Audit trail	AuditService on every mutation, audit log UI, CSV export
11	Security hardening	Rate limiting, security headers, pentest, dependency audit
12	Observability + Launch	Sentry, structured logging, health checks, production deploy

Success Criteria:

- 10,000 families can use the app simultaneously without degradation
- Every mutation is audited + persists to PostgreSQL
- Every route is protected by auth + RBAC
- Every file upload goes through presigned S3 URLs (not base64 in DB)
- Every notification can be delivered via in-app + email + push
- Every report can be generated + emailed as PDF/CSV
- Every background job is idempotent + retry-safe
- Zero P0 security vulnerabilities (verified by pentest)